Estado, Memória e Composição de Programas

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Estado, Memória e Estrutura

2. Ler Programas por Estados

Um Programa Tem Estrutura

- É verdade que, enquanto texto, um programa é uma sequência de “linhas de código” … Mas ver um programa como uma sequência de linhas de código é profundamente errado!

Um Programa Tem Estrutura

- É verdade que, enquanto texto, um programa é uma sequência de “linhas de código” … Mas ver um programa como uma sequência de linhas de código é profundamente errado!
- Programas bem construídos têm uma estrutura “em partes” interligadas facilmente reconhecível, simples e funcional.

Um Programa Tem Estrutura

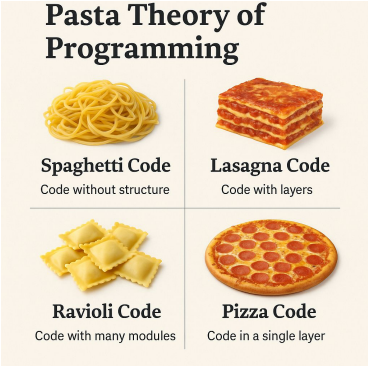
- É verdade que, enquanto texto, um programa é uma sequência de “linhas de código” … Mas ver um programa como uma sequência de linhas de código é profundamente errado!
- Programas bem construídos têm uma estrutura “em partes” interligadas facilmente reconhecível, simples e funcional.
- Nesta aula vamos reconhecer duas formas de composição:
  - passos executados em **sequência**;
  - caminhos escolhidos em **alternativa**.

Um Programa Tem Estrutura

- É verdade que, enquanto texto, um programa é uma sequência de “linhas de código” … Mas ver um programa como uma sequência de linhas de código é profundamente errado!
- Programas bem construídos têm uma estrutura “em partes” interligadas facilmente reconhecível, simples e funcional.
- Nesta aula vamos reconhecer duas formas de composição:
  - passos executados em **sequência**;
  - caminhos escolhidos em **alternativa**.
- Para compreender cada passo, acompanhamos o **estado da memória**: os valores associados aos nomes naquele momento.

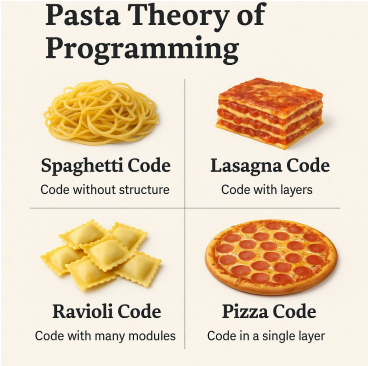
“Ingredientes” das Linguagens de Programação

- Todas as linguagens de programação imperativas modernas aderem aos princípios da “programação estruturada” [Dijkstra66]



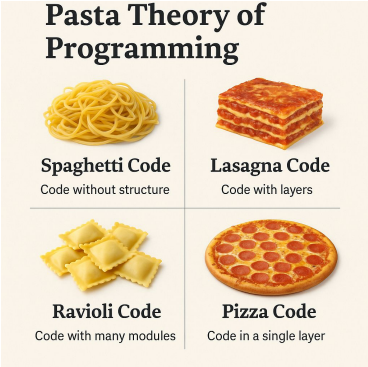
“Ingredientes” das Linguagens de Programação

- Todas as linguagens de programação imperativas modernas aderem aos princípios da “programação estruturada” [Dijkstra66]
- Os programas são construídos com base nas seguintes construções fundamentais:
  - **Elementos Básicos**
    - expressões (calculam um valor)
    - comandos básicos (executam uma ação)
  - Subprogramas em Sequência
  - Subprogramas em Alternativa
  - Subprogramas em Repetição (a partir da Aula 5)



“Ingredientes” das Linguagens de Programação

- Todas as linguagens de programação imperativas modernas aderem aos princípios da “programação estruturada” [Dijkstra66]
- Os programas são construídos com base nas seguintes construções fundamentais:
  - **Elementos Básicos**
    - expressões (calculam um valor)
    - comandos básicos (executam uma ação)
  - Subprogramas em Sequência
  - Subprogramas em Alternativa
  - Subprogramas em Repetição (a partir da Aula 5)
- Um programa **não** deve ser uma lista de “instruções” e “decisões” ligadas por caminhos alternativos de forma confusa e monolítica…



Programação Estruturada: A Ideia de Dijkstra

- Nos anos 1960, muitos programas eram escritos usando **GOTO**: saltos arbitrários para qualquer ponto do código.

Programação Estruturada: A Ideia de Dijkstra

- Nos anos 1960, muitos programas eram escritos usando **GOTO**: saltos arbitrários para qualquer ponto do código.
- Em 1968, Dijkstra publicou "Go To Statement Considered Harmful" — um dos artigos mais influentes da Computação.

Nota. O termo "spaghetti code" descreve código com saltos de controlo difíceis de seguir — como um prato de esparguete.

Programação Estruturada: A Ideia de Dijkstra

- Nos anos 1960, muitos programas eram escritos usando **GOTO**: saltos arbitrários para qualquer ponto do código.
- Em 1968, Dijkstra publicou "Go To Statement Considered Harmful" — um dos artigos mais influentes da Computação.
- A tese: qualquer programa pode ser escrito **sem GOTO**, combinando sequência, alternativa e iteração.

Nota. O termo "spaghetti code" descreve código com saltos de controlo difíceis de seguir — como um prato de esparguete.

Programação Estruturada: A Ideia de Dijkstra

- Nos anos 1960, muitos programas eram escritos usando **GOTO**: saltos arbitrários para qualquer ponto do código.
- Em 1968, Dijkstra publicou "Go To Statement Considered Harmful" — um dos artigos mais influentes da Computação.
- A tese: qualquer programa pode ser escrito **sem GOTO**, combinando sequência, alternativa e iteração.
- Benefício: programas estruturados são **compreensíveis**, **verificáveis** e **manuteníveis**.

Nota. O termo "spaghetti code" descreve código com saltos de controlo difíceis de seguir — como um prato de esparguete.

Ideia-chave. O teorema de Böhm-Jacopini mostra que sequência, alternativa e iteração são suficientes para estruturar programas.

Programação Estruturada: A Ideia de Dijkstra

- Nos anos 1960, muitos programas eram escritos usando **GOTO**: saltos arbitrários para qualquer ponto do código.
- Em 1968, Dijkstra publicou "Go To Statement Considered Harmful" — um dos artigos mais influentes da Computação.
- A tese: qualquer programa pode ser escrito **sem GOTO**, combinando sequência, alternativa e iteração.
- Benefício: programas estruturados são **compreensíveis**, **verificáveis** e **manuteníveis**.
- Python **não tem** instrução GOTO — força boas práticas.

Nota. O termo "spaghetti code" descreve código com saltos de controlo difíceis de seguir — como um prato de esparguete.

Ideia-chave. O teorema de Böhm-Jacopini mostra que sequência, alternativa e iteração são suficientes para estruturar programas.

Do Problema ao Código

Problema: autorizar uma compra apenas quando o preço é válido e o saldo é suficiente.

Pseudocódigo:

1. Ler o saldo e o preço
2. Se o preço não for positivo, indicar erro
3. Caso contrário, se houver saldo, calcular o novo saldo
4. Senão, indicar saldo insuficiente

Do Problema ao Código

Problema: autorizar uma compra apenas quando o preço é válido e o saldo é suficiente.

Pseudocódigo:

1. Ler o saldo e o preço
2. Se o preço não for positivo, indicar erro
3. Caso contrário, se houver saldo, calcular o novo saldo
4. Senão, indicar saldo insuficiente

- Primeiro separam-se as decisões e os resultados; só depois se escolhe a sintaxe.

Do Problema ao Código

Problema: autorizar uma compra apenas quando o preço é válido e o saldo é suficiente.

Pseudocódigo:

1. Ler o saldo e o preço
2. Se o preço não for positivo, indicar erro
3. Caso contrário, se houver saldo, calcular o novo saldo
4. Senão, indicar saldo insuficiente

Código Python:

```
1  saldo = float(input("Saldo: "))
2  preco = float(input("Preco: "))
3
4  if preco <= 0:
5      print("preco invalido")
6  elif preco <= saldo:
7      saldo = saldo - preco
8      print("novo saldo:", saldo)
9  else:
10     print("saldo insuficiente")
```

- Primeiro separam-se as decisões e os resultados; só depois se escolhe a sintaxe.
- A ordem dos casos no código segue a ordem do raciocínio.

Precedência e Avaliação de Expressões

- A **precedência** determina a ordem de avaliação dos operadores.

```
2 + 3 * 4        # 14  (nao 20)
(2 + 3) * 4      # 20
2 ** 3 ** 2      # 512 (2**(3**2), dir p/ esq)
(2 ** 3) ** 2    # 64
-2 ** 2          # -4  (nao 4: unario apos **)
(-2) ** 2        # 4
10 / 2 * 5       # 25.0 (esq p/ dir)
```

Precedência e Avaliação de Expressões

- A **precedência** determina a ordem de avaliação dos operadores.
- Maior para menor: `**` > unários (`-x`) > `*`, `/`, `//`, `%` > `+`, `-` > `<`, `>`, `==`, `!=` > **`not`** > **`and`** > **`or`**.

```
2 + 3 * 4          # 14  (nao 20)
(2 + 3) * 4        # 20
2 ** 3 ** 2        # 512 (2**(3**2), dir p/ esq)
(2 ** 3) ** 2      # 64
-2 ** 2            # -4  (nao 4: unario apos **)
(-2) ** 2          # 4
10 / 2 * 5         # 25.0 (esq p/ dir)
```

Dica

Em expressões complexas, adiciona parênteses mesmo quando não são necessários — o código fica mais legível e evita bugs.

```
# Difícil de ler:
x = a + b * c - d / e ** 2

# Mais claro:
x = a + (b * c) - (d / (e ** 2))
```

Precedência e Avaliação de Expressões

- A **precedência** determina a ordem de avaliação dos operadores.
- Maior para menor: `**` > unários (`-x`) > `*`, `/`, `//`, `%` > `+`, `-` > `<`, `>`, `==`, `!=` > **`not`** > **`and`** > **`or`**.
- Quando em dúvida, usar **parênteses** para clareza.

```
2 + 3 * 4        # 14  (nao 20)
(2 + 3) * 4      # 20
2 ** 3 ** 2      # 512 (2**(3**2), dir p/ esq)
(2 ** 3) ** 2    # 64
-2 ** 2          # -4  (nao 4: unario apos **)
(-2) ** 2        # 4
10 / 2 * 5       # 25.0 (esq p/ dir)
```

Dica

Em expressões complexas, adiciona parênteses mesmo quando não são necessários — o código fica mais legível e evita bugs.

```
# Difícil de ler:
x = a + b * c - d / e ** 2

# Mais claro:
x = a + (b * c) - (d / (e ** 2))
```

Avaliar Expressões Passo a Passo

Exemplo: `(a + b)* c - d % 2 == 0` com `a=2, b=3, c=4, d=9`

1. `a + b` $\rightarrow$ `5`

Avaliar Expressões Passo a Passo

Exemplo: `(a + b)* c - d % 2 == 0` com `a=2, b=3, c=4, d=9`

1. `a + b` → `5`
2. `5 * c` → `20`

Avaliar Expressões Passo a Passo

Exemplo: `(a + b)* c - d % 2 == 0` com `a=2, b=3, c=4, d=9`

1. `a + b` → `5`
2. `5 * c` → `20`
3. `d % 2` → `1`

Avaliar Expressões Passo a Passo

Exemplo: `(a + b)* c - d % 2 == 0` com `a=2, b=3, c=4, d=9`

1. `a + b` → `5`
2. `5 * c` → `20`
3. `d % 2` → `1`
4. `20 - 1` → `19`

Avaliar Expressões Passo a Passo

Exemplo: `(a + b)* c - d % 2 == 0` com `a=2, b=3, c=4, d=9`

1. `a + b` → `5`
2. `5 * c` → `20`
3. `d % 2` → `1`
4. `20 - 1` → `19`
5. `19 == 0` → **`False`**

Avaliar Expressões Passo a Passo

Exemplo: `(a + b)* c - d % 2 == 0` com `a=2, b=3, c=4, d=9`

1. `a + b` → `5`
2. `5 * c` → `20`
3. `d % 2` → `1`
4. `20 - 1` → `19`
5. `19 == 0` → **`False`**

```python
a, b, c, d = 2, 3, 4, 9

# Passo a passo:
step1 = a + b         # 5
step2 = step1 * c     # 20
step3 = d % 2         # 1
step4 = step2 - step3 # 19
step5 = step4 == 0    # False
```

- Python avalia sempre as sub-expressões **da esquerda para a direita** dentro do mesmo nível de precedência.

Afetação

- O comando básico mais importante, que define a linguagem imperativa, é a afetação:

```
1 X = EXP
```

Programação Imperativa

Na programação imperativa uma "variável" representa uma célula de memória cujo conteúdo pode ser alterado usando a afetação.

Em Python, uma variável é criada no momento em que é executada a primeira afetação ao nome escolhido.

Afetação

- O comando básico mais importante, que define a linguagem imperativa, é a afetação:

```
1 X = EXP
```

- A afetação altera o conteúdo da variável `X` à esquerda de `=` para o valor de `EXP`.

Programação Imperativa

Na programação imperativa uma "variável" representa uma célula de memória cujo conteúdo pode ser alterado usando a afetação.

Em Python, uma variável é criada no momento em que é executada a primeira afetação ao nome escolhido.

Afetação

- O comando básico mais importante, que define a linguagem imperativa, é a afetação:

```
1 X = EXP
```

- A afetação altera o conteúdo da variável `X` à esquerda de `=` para o valor de `EXP`.
- O comando afetação é executado em 2 passos:
  1. a expressão `EXP` é avaliada, produzindo um valor V
  2. esse valor V é armazenado na variável `X`

Programação Imperativa

Na programação imperativa uma "variável" representa uma célula de memória cujo conteúdo pode ser alterado usando a afetação.

Em Python, uma variável é criada no momento em que é executada a primeira afetação ao nome escolhido.

Afetações e Memória

- O **estado da memória** é o conjunto de valores atualmente associados às variáveis.

Antes

variável	valor
a	3
b	5

Afetações e Memória

- O **estado da memória** é o conjunto de valores atualmente associados às variáveis.
- Avaliar uma expressão, como `a + b`, **não altera** esse estado: apenas produz um valor.

Antes

variável	valor
a	3
b	5

`a + b` $\longrightarrow$ 8
sem mudar `a` ou `b`

Afetações e Memória

- O **estado da memória** é o conjunto de valores atualmente associados às variáveis.
- Avaliar uma expressão, como `a + b`, **não altera** esse estado: apenas produz um valor.
- Executar uma afetação, como `a = a + b`, **atualiza** a variável à esquerda.

Antes

variável	valor
a	3
b	5

`a + b` $\longrightarrow$ 8
sem mudar `a` ou `b`

Depois de `a = a + b`

variável	valor
a	8
b	5

Afetações e Memória

- O **estado da memória** é o conjunto de valores atualmente associados às variáveis.
- Avaliar uma expressão, como `a + b`, **não altera** esse estado: apenas produz um valor.
- Executar uma afetação, como `a = a + b`, **atualiza** a variável à esquerda.

Regra mental: primeiro calcula-se o lado direito; só depois se escreve o resultado no nome do lado esquerdo.

Antes

variável	valor
a	3
b	5

`a + b` $\longrightarrow$ 8

sem mudar `a` ou `b`

Depois de `a = a + b`

variável	valor
a	8
b	5

(CAVEAT!) Afetação e Igualdade

- A afetação (`X = E`) **não** deve ser confundida com a igualdade (`E == E`).
- A afetação é uma operação **assimétrica**, que altera o estado da variável à esquerda.
- A igualdade (`E == E`) é uma operação **simétrica** que verifica se 2 valores são iguais, devolvendo um valor booleano (`True` ou `False`).

Trace de Afetação na Memória

Trace do programa:

```
1 x = 5          # x -> 5
2 y = x + 1      # y -> 6
3 x = x * 2      # x -> 10  (y nao muda!)
4 y = x - y      # y -> 4
```

Linha	x	y

Trace de Afetação na Memória

Trace do programa:

```
1 x = 5          # x -> 5
2 y = x + 1      # y -> 6
3 x = x * 2    # x -> 10  (y nao muda!)
4 y = x - y    # y -> 4
```

Linha	x	y
x = 5	5	—

- Cada afetação **destrói** o valor anterior da variável.

Trace de Afetação na Memória

Trace do programa:

```
1 x = 5          # x -> 5
2 y = x + 1      # y -> 6
3 x = x * 2      # x -> 10  (y nao muda!)
4 y = x - y      # y -> 4
```

Linha	x	y
x = 5	5	—
y = x + 1	5	6

- Cada afetação **destrói** o valor anterior da variável.
- O valor de outras variáveis **não muda** automaticamente.

Trace de Afetação na Memória

Trace do programa:

```
1 x = 5          # x -> 5
2 y = x + 1      # y -> 6
3 x = x * 2    # x -> 10  (y nao muda!)
4 y = x - y    # y -> 4
```

Linha	x	y
`x = 5`	5	—
`y = x + 1`	5	6
`x = x * 2`	10	6

- Cada afetação **destrói** o valor anterior da variável.
- O valor de outras variáveis **não muda** automaticamente.
- A expressão à direita é avaliada com os **valores actuais** antes de actualizar.

Trace de Afetação na Memória

Trace do programa:

```
1 x = 5          # x -> 5
2 y = x + 1      # y -> 6
3 x = x * 2      # x -> 10  (y nao muda!)
4 y = x - y      # y -> 4
```

Linha	x	y
`x = 5`	5	—
`y = x + 1`	5	6
`x = x * 2`	10	6
`y = x - y`	10	4

- Cada afetação **destrói** o valor anterior da variável.
- O valor de outras variáveis **não muda** automaticamente.
- A expressão à direita é avaliada com os **valores actuais** antes de actualizar.
- Construir traces de memória é essencial para compreender o que um programa faz.

Afetação Composta

- Python oferece atalhos para o padrão `x = x op expr`:

Atalho	Equivalente
`x += 3`	`x = x + 3`
`x -= 2`	`x = x - 2`
`x *= 4`	`x = x * 4`
`x /= 2`	`x = x / 2`
`x //= 3`	`x = x // 3`
`x %= 5`	`x = x % 5`
`x **= 2`	`x = x ** 2`

Afetação Composta

- Python oferece atalhos para o padrão `x = x op expr`:

Atalho	Equivalente
`x += 3`	`x = x + 3`
`x -= 2`	`x = x - 2`
`x *= 4`	`x = x * 4`
`x /= 2`	`x = x / 2`
`x //= 3`	`x = x // 3`
`x %= 5`	`x = x % 5`
`x **= 2`	`x = x ** 2`

```
1 saldo = 40.0
2 saldo -= 12.5   # saldo = 27.5
3
4 pontos = 3
5 pontos += 5     # pontos = 8
6 pontos *= 2     # pontos = 16
7
8 paginas = 17
9 paginas //= 2   # paginas = 8
```

Afetação Composta

- Python oferece atalhos para o padrão `x = x op expr`:

Atalho	Equivalente
`x += 3`	`x = x + 3`
`x -= 2`	`x = x - 2`
`x *= 4`	`x = x * 4`
`x /= 2`	`x = x / 2`
`x //= 3`	`x = x // 3`
`x %= 5`	`x = x % 5`
`x **= 2`	`x = x ** 2`

```
1 saldo = 40.0
2 saldo -= 12.5   # saldo = 27.5
3
4 pontos = 3
5 pontos += 5     # pontos = 8
6 pontos *= 2     # pontos = 16
7
8 paginas = 17
9 paginas //= 2   # paginas = 8
```

- A forma composta continua a ser uma **afetação**: primeiro avalia, depois atualiza.

Afetação Composta

- Python oferece atalhos para o padrão `x = x op expr`:

Atalho	Equivalente
`x += 3`	`x = x + 3`
`x -= 2`	`x = x - 2`
`x *= 4`	`x = x * 4`
`x /= 2`	`x = x / 2`
`x //= 3`	`x = x // 3`
`x %= 5`	`x = x % 5`
`x **= 2`	`x = x ** 2`

```
1 saldo = 40.0
2 saldo -= 12.5   # saldo = 27.5
3
4 pontos = 3
5 pontos += 5     # pontos = 8
6 pontos *= 2     # pontos = 16
7
8 paginas = 17
9 paginas //= 2   # paginas = 8
```

- A forma composta continua a ser uma **afetação**: primeiro avalia, depois atualiza.
- O valor anterior participa no cálculo, mas é substituído pelo novo valor.

Composição Sequencial

- A composição sequencial representa a execução dos subprogramas em sequência.



Composição Sequencial

- A composição sequencial representa a execução dos subprogramas em sequência.
- É executada da seguinte forma:
  1. P_1 é executado.
  2. Quando P_1 terminar, é executado P_2.




```
x = 2
y = x + 3
```

Composição Sequencial

- A composição sequencial representa a execução dos subprogramas em sequência.
- É executada da seguinte forma:
  1. P_1 é executado.
  2. Quando P_1 terminar, é executado P_2.
- Permite decompor um processo complexo numa sequência de passos sucessivos mais simples.




```
x = 2
y = x + 3
```

passo	x	y
início	—	—
x = 2	2	—
y = x + 3	2	5

Composição Sequencial

- A composição sequencial representa a execução dos subprogramas em sequência.
- É executada da seguinte forma:
  1. P_1 é executado.
  2. Quando P_1 terminar, é executado P_2.
- Permite decompor um processo complexo numa sequência de passos sucessivos mais simples.
- Se P_1 não terminar, P_2 nunca chega a ser executado.




```
x = 2
y = x + 3
```

passo	x	y
início	—	—
x = 2	2	—
y = x + 3	2	5

A segunda instrução pode usar o estado produzido pela primeira.

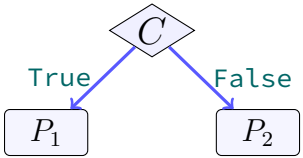
Composição Alternativa

- A composição alternativa representa a execução em alternativa, determinada por uma condição C (expressão booleana).



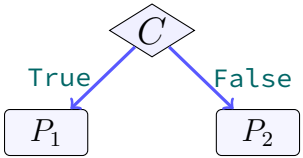
Composição Alternativa

- A composição alternativa representa a execução em alternativa, determinada por uma condição C (expressão booleana).
- É executada da seguinte forma:
  1. A condição C é avaliada.
  2. Se C for verdadeira (`True`), será executado P_1.
  3. Se C for falsa (`False`), será executado P_2.



Composição Alternativa

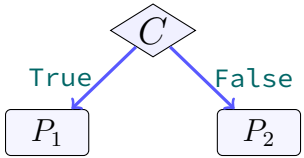
- A composição alternativa representa a execução em alternativa, determinada por uma condição C (expressão booleana).
- É executada da seguinte forma:
  1. A condição C é avaliada.
  2. Se C for verdadeira (`True`), será executado P_1.
  3. Se C for falsa (`False`), será executado P_2.
- Apenas um de P_1 ou P_2 será executado.




```
x = 7
if x >= 0:
    sinal = "positivo"
else:
    sinal = "negativo"
```

Composição Alternativa

- A composição alternativa representa a execução em alternativa, determinada por uma condição C (expressão booleana).
- É executada da seguinte forma:
  1. A condição C é avaliada.
  2. Se C for verdadeira (`True`), será executado P_1.
  3. Se C for falsa (`False`), será executado P_2.
- Apenas um de P_1 ou P_2 será executado.
- Permite que um programa escolha ações alternativas com base no seu estado.




```
x = 7
if x >= 0:
    sinal = "positivo"
else:
    sinal = "negativo"
```

teste	ramo
`True`	`sinal = "positivo"`
`False`	`sinal = "negativo"`

Condicionais Aninhados

```
# Aninhado (evitar quando possivel)
if condicao1:
    if condicao2:
        instrucoes_A
    else:
        instrucoes_B
else:
    instrucoes_C

# Preferir elif (mais plano):
if cond1 and cond2:
    instrucoes_A
elif cond1:
    instrucoes_B
else:
    instrucoes_C
```

Condicionais Aninhados

```python
# Aninhado (evitar quando possivel)
if condicao1:
    if condicao2:
        instrucoes_A
    else:
        instrucoes_B
else:
    instrucoes_C

# Preferir elif (mais plano):
if cond1 and cond2:
    instrucoes_A
elif cond1:
    instrucoes_B
else:
    instrucoes_C
```

- Condicionais muito aninhados são difíceis de ler — maximizar o uso de `elif`.

Condicionais Aninhados

```python
# Aninhado (evitar quando possivel)
if condicao1:
    if condicao2:
        instrucoes_A
    else:
        instrucoes_B
else:
    instrucoes_C

# Preferir elif (mais plano):
if cond1 and cond2:
    instrucoes_A
elif cond1:
    instrucoes_B
else:
    instrucoes_C
```

- Condicionais muito aninhados são difíceis de ler — maximizar o uso de `elif`.
- **Early return**: em funções, devolver cedo evita aninhamento.

```python
# Com aninhamento:
def categoria(n):
    if n >= 0:
        if n < 10:
            return "pequeno"
        return "grande"
    return "negativo"

# Com early return (mais claro):
def categoria(n):
    if n < 0: return "negativo"
    if n < 10: return "pequeno"
    return "grande"
```

Condicionais: Exemplo Classificação de Nota

```python
def classificar_nota(nota: float) -> str:
    """Classifica uma nota de 0 a 20."""
    if nota < 0 or nota > 20:
        return "Nota inválida"
    elif nota < 10:
        return "Reprovado"
    elif nota < 14:
        return "Suficiente"
    elif nota < 16:
        return "Bom"
    elif nota < 18:
        return "Muito Bom"
    else:
        return "Excelente"

# Testes:
print(classificar_nota(9.9))   # Reprovado
print(classificar_nota(10))    # Suficiente
print(classificar_nota(18))    # Excelente
print(classificar_nota(21))    # Nota inválida
```

Condicionais: Exemplo Classificação de Nota

```python
def classificar_nota(nota: float) -> str:
    """Classifica uma nota de 0 a 20."""
    if nota < 0 or nota > 20:
        return "Nota inválida"
    elif nota < 10:
        return "Reprovado"
    elif nota < 14:
        return "Suficiente"
    elif nota < 16:
        return "Bom"
    elif nota < 18:
        return "Muito Bom"
    else:
        return "Excelente"

# Testes:
print(classificar_nota(9.9))   # Reprovado
print(classificar_nota(10))    # Suficiente
print(classificar_nota(18))    # Excelente
print(classificar_nota(21))    # Nota inválida
```

- Teste **do caso de erro primeiro** (nota fora do intervalo) antes dos casos normais — padrão “guard clause”.

Condicionais: Expressão Condicional (Ternary)

- Python tem uma forma compacta de escrever um `if`/`else` numa única expressão.

```python
# Forma longa:
if a > b:
    maximo = a
else:
    maximo = b

# Forma compacta (ternary):
maximo = a if a > b else b

# Mais exemplos:
sinal = "+" if x >= 0 else "-"
abs_x = x if x >= 0 else -x
par = "par" if n % 2 == 0 else "impar"

# Pode usar em expressoes maiores:
print(f"n é {par}")
```

Condicionais: Expressão Condicional (Ternary)

- Python tem uma forma compacta de escrever um `if`/`else` numa única expressão.

```python
# Forma longa:
if a > b:
    maximo = a
else:
    maximo = b

# Forma compacta (ternary):
maximo = a if a > b else b

# Mais exemplos:
sinal = "+" if x >= 0 else "-"
abs_x = x if x >= 0 else -x
par = "par" if n % 2 == 0 else "impar"

# Pode usar em expressoes maiores:
print(f"n é {par}")
```

- Usar apenas para expressões **simples** — condicionais complexos devem usar a forma longa para não perder legibilidade.

Condicionais: Expressão Condicional (Ternary)

- Python tem uma forma compacta de escrever um `if`/`else` numa única expressão.

```python
# Forma longa:
if a > b:
    maximo = a
else:
    maximo = b

# Forma compacta (ternary):
maximo = a if a > b else b

# Mais exemplos:
sinal = "+" if x >= 0 else "-"
abs_x = x if x >= 0 else -x
par = "par" if n % 2 == 0 else "impar"

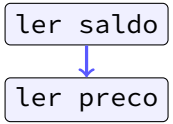
# Pode usar em expressoes maiores:
print(f"n é {par}")
```

- Usar apenas para expressões **simples** — condicionais complexos devem usar a forma longa para não perder legibilidade.
- Estrutura:

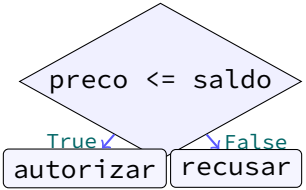
```
VALOR_SE_TRUE if COND else VALOR_SE_FALSE
```

Decompor um Programa em Etapas

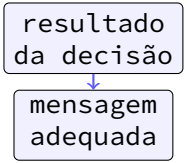
1. Obter dados



2. Escolher um caso

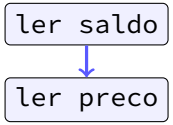


3. Comunicar



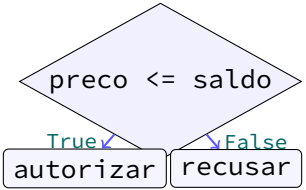
Decompor um Programa em Etapas

1. Obter dados

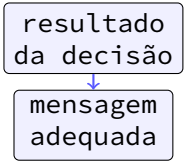


O segundo passo ocorre depois do primeiro.

2. Escolher um caso

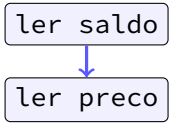


3. Comunicar



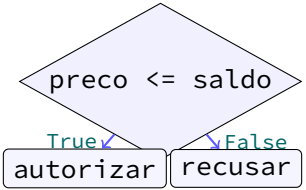
Decompor um Programa em Etapas

1. Obter dados



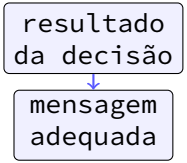
O segundo passo ocorre depois do primeiro.

2. Escolher um caso



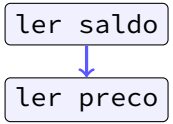
Só um dos caminhos altera o saldo.

3. Comunicar



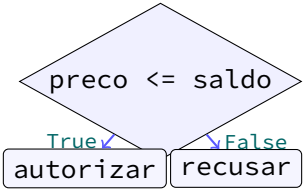
Decompor um Programa em Etapas

1. Obter dados



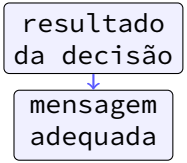
O segundo passo ocorre depois do primeiro.

2. Escolher um caso



Só um dos caminhos altera o saldo.

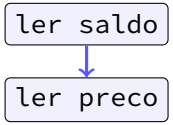
3. Comunicar



A saída deve explicar o caso que ocorreu.

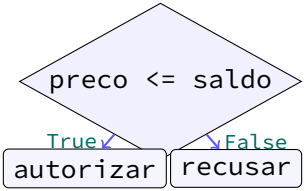
Decompor um Programa em Etapas

1. Obter dados



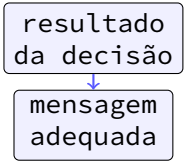
O segundo passo ocorre depois do primeiro.

2. Escolher um caso



Só um dos caminhos altera o saldo.

3. Comunicar



A saída deve explicar o caso que ocorreu.

A estrutura resulta da ligação explícita entre passos e decisões.

Outline

1. Estado, Memória e Estrutura

2. Ler Programas por Estados

Estado à Entrada e à Saída de uma Decisão

```
1  saldo = 40
2  preco = 12
3
4  if preco <= saldo:
5      saldo = saldo - preco
6      mensagem = "autorizada"
7  else:
8      mensagem = "recusada"
9
10 print(mensagem, saldo)
```

- A condição é avaliada com o estado **antes** do ramo.

Estado à Entrada e à Saída de uma Decisão

```
1  saldo = 40
2  preco = 12
3
4  if preco <= saldo:
5      saldo = saldo - preco
6      mensagem = "autorizada"
7  else:
8      mensagem = "recusada"
9
10 print(mensagem, saldo)
```

- A condição é avaliada com o estado **antes** do ramo.
- Só o ramo escolhido pode alterar esse estado.

Estado à Entrada e à Saída de uma Decisão

```
1  saldo = 40
2  preco = 12
3
4  if preco <= saldo:
5      saldo = saldo - preco
6      mensagem = "autorizada"
7  else:
8      mensagem = "recusada"
9
10 print(mensagem, saldo)
```

- A condição é avaliada com o estado **antes** do ramo.
- Só o ramo escolhido pode alterar esse estado.
- A execução continua na primeira instrução depois do `if`.

Estado à Entrada e à Saída de uma Decisão

```
1  saldo = 40
2  preco = 12
3
4  if preco <= saldo:
5      saldo = saldo - preco
6      mensagem = "autorizada"
7  else:
8      mensagem = "recusada"
9
10 print(mensagem, saldo)
```

- A condição é avaliada com o estado **antes** do ramo.
- Só o ramo escolhido pode alterar esse estado.
- A execução continua na primeira instrução depois do `if`.
- Neste caso, o estado final contém `saldo == 28` e `mensagem == "autorizada"`.

Exercício: Dois Caminhos, Dois Traces

Programa

```
1 if x > 2 * y:
2     x = x - y
3 else:
4     y = y + x
5 z = x + y
```

Questão

Construa o trace de `x`, `y` e `z` para os estados iniciais `x=8, y=3` e `x=4, y=3`. Que instruções não são executadas em cada caso?

Resultado Esperado: Dois Caminhos, Dois Traces

Estado inicial: `x=8, y=3`

passo	x	y	z
início	8	3	—
ramo `if`	5	3	—
depois do `if`	5	3	8

Estado inicial: `x=4, y=3`

passo	x	y	z
início	4	3	—
ramo `else`	4	7	—
depois do `if`	4	7	11

Em ambos os casos, a execução reencontra-se em `z = x + y`.

Depois de uma Alternativa

```
1 if temperatura < 18:
2     estado = "frio"
3 else:
4     estado = "ameno"
5
6 print("Estado:", estado)
```

- Os ramos são alternativos, mas o programa continua depois deles.

Depois de uma Alternativa

```
1 if temperatura < 18:
2     estado = "frio"
3 else:
4     estado = "ameno"
5
6 print("Estado:", estado)
```

- Os ramos são alternativos, mas o programa continua depois deles.
- Se uma variável for usada depois do `if`, deve receber valor em **todos** os caminhos relevantes.

Depois de uma Alternativa

```
1 if temperatura < 18:
2     estado = "frio"
3 else:
4     estado = "ameno"
5
6 print("Estado:", estado)
```

- Os ramos são alternativos, mas o programa continua depois deles.
- Se uma variável for usada depois do `if`, deve receber valor em **todos** os caminhos relevantes.
- Pensar nos caminhos evita erros como usar um nome que só foi criado num dos ramos.

Um Erro de Estado entre Ramos

```
1 if pontos >= 10:
2     nivel = "avancado"
3
4 print(nivel)
```

- Para `pontos >= 10`, o programa define e imprime `nivel`.

Um Erro de Estado entre Ramos

```
1 if pontos >= 10:
2     nivel = "avancado"
3
4 print(nivel)
```

- Para `pontos >= 10`, o programa define e imprime `nivel`.
- Para `pontos < 10`, a afetação não é executada.

Um Erro de Estado entre Ramos

```
1 if pontos >= 10:
2     nivel = "avancado"
3
4 print(nivel)
```

- Para `pontos >= 10`, o programa define e imprime `nivel`.
- Para `pontos < 10`, a afetação não é executada.
- Nesse caminho, `nivel` ainda não existe quando chega ao **`print`**.

Um Erro de Estado entre Ramos

```
1 if pontos >= 10:
2     nivel = "avancado"
3
4 print(nivel)
```

- Para `pontos >= 10`, o programa define e imprime `nivel`.
- Para `pontos < 10`, a afetação não é executada.
- Nesse caminho, `nivel` ainda não existe quando chega ao **`print`**.

Correção estrutural

Adicionar um ramo **`else`** que também atribua um valor a `nivel`.

Exercício: Ordenar as Decisões

Problema

Um levantamento recebe um `valor`, um `saldo` e tem limite de 100 euros. Deve produzir exatamente uma das mensagens seguintes:

- `"valor invalido"`, se `valor <= 0`;
- `"acima do limite"`, se `valor > 100`;
- `"saldo insuficiente"`, se `valor > saldo`;
- `"autorizado"`, nos restantes casos.

Questão

Escreva a estrutura **`if`**/**`elif`**/**`else`**. Cada caso deve ser tratado uma única vez.

Comportamento Esperado: Ordenar as Decisões

valor	saldo	condição decisiva	resultado
-5	200	`valor <= 0`	valor inválido
120	200	`valor > 100`	acima do limite
80	50	`valor > saldo`	saldo insuficiente
80	200	nenhuma anterior	autorizado

Os casos específicos e inválidos são testados antes do caso geral.

Uma Solução Possível: Ordenar as Decisões

Problema: classificar um levantamento usando o valor, o saldo e o limite de 100 euros.

```
1  if valor <= 0:
2      mensagem = "valor invalido"
3  elif valor > 100:
4      mensagem = "acima do limite"
5  elif valor > saldo:
6      mensagem = "saldo insuficiente"
7  else:
8      mensagem = "autorizado"
9
10 print(mensagem)
```

Testar Todos os Caminhos

- Um exemplo bem escolhido por ramo confirma que cada caminho é alcançável.

Testar Todos os Caminhos

- Um exemplo bem escolhido por ramo confirma que cada caminho é alcançável.
- Os valores junto às fronteiras revelam erros em <, <=, > e >=.

Testar Todos os Caminhos

- Um exemplo bem escolhido por ramo confirma que cada caminho é alcançável.
- Os valores junto às fronteiras revelam erros em `<`, `<=`, `>` e `>=`.
- Para o limite de 100 euros, devemos distinguir pelo menos `99`, `100` e `101`.

Testar Todos os Caminhos

- Um exemplo bem escolhido por ramo confirma que cada caminho é alcançável.
- Os valores junto às fronteiras revelam erros em `<`, `<=`, `>` e `>=`.
- Para o limite de 100 euros, devemos distinguir pelo menos `99`, `100` e `101`.
- Um teste não substitui o raciocínio: ajuda a confrontar a previsão com a execução.

Testar Todos os Caminhos

- Um exemplo bem escolhido por ramo confirma que cada caminho é alcançável.
- Os valores junto às fronteiras revelam erros em `<`, `<=`, `>` e `>=`.
- Para o limite de 100 euros, devemos distinguir pelo menos `99`, `100` e `101`.
- Um teste não substitui o raciocínio: ajuda a confrontar a previsão com a execução.

Regra prática

Para cada ramo, escreva um caso que entra nele e confirme o estado produzido à saída.

Exercício: Diagnóstico de Estado

Ponto de partida

Sem executar: `x=2; y=5; x=x+y; y=x-y; x=x-y`.

Questão

Preveja o resultado e justifique a resposta antes de executar.

Resultado Esperado: Diagnóstico de Estado

instrução executada	x	y
`x = 2; y = 5`	2	5
`x = x + y`	7	5
`y = x - y`	7	2
`x = x - y`	5	2

Explicação

Cada lado direito usa o estado atual. Os valores iniciais ficam temporariamente codificados em `x == 7` e depois são recuperados pelas subtrações.

Resumo da Aula 3

- Como abordar um problema de programação:
  - Compreender o problema
  - Imaginar como se resolve num caso concreto
  - Pensar o algoritmo geral em termos esquemáticos (pseudo código)
  - Codificar usando a linguagem de programação
- Estruturas básicas da programação imperativa:
  - Comandos básicos, expressões, afetações
  - Comando alternativo (`if then else`, `elif`)
  - Composição sequencial
  - Composição alternativa (`if`, `elif`, `else`)
  - Trace do estado pelos vários caminhos de execução

Exercício: Seguir uma Troca Condicional

```
1 a = 4
2 b = 9
3 if a < b:
4     auxiliar = a
5     a = b
6     b = auxiliar
7 diferenca = a - b
```

Questão

Qual é o estado depois de cada afetação? O que muda se os valores iniciais forem `a=12` e `b=5`?

Resposta: Seguir uma Troca Condicional

- Para `a=4, b=9`, a condição é verdadeira: no fim, `a=9`, `b=4` e `diferenca=5`.
- Para `a=12, b=5`, o ramo não executa: no fim, `a=12`, `b=5` e `diferenca=7`.
- Em ambos os casos, `a >= b` quando se calcula a diferença.

Exercício: Encontrar o Caminho com Erro

```
1 if quantidade > 0:
2     custo = quantidade * preco
3 elif quantidade == 0:
4     mensagem = "sem artigos"
5
6 print(custo)
```

Questão

Para que valores de `quantidade` o programa chega ao **`print`** sem ter definido `custo`? Como pode garantir-se um estado válido em todos os caminhos?

Resposta: Encontrar o Caminho com Erro

- Para `quantidade == 0`, só é definida `mensagem`.
- Para `quantidade < 0`, nenhum dos ramos é executado.
- Portanto, em ambos os caminhos `custo` não existe no **`print`**.

```
1 if quantidade > 0:
2     custo = quantidade * preco
3 else:
4     custo = 0
5 print(custo)
```

Exercício: Especificar Antes de Programar

Problema

Calcular o custo de estacionamento a partir do número inteiro de horas:

- horas negativas são inválidas;
- zero horas custam 0 euros;
- até 2 horas custam 3 euros;
- mais de 2 horas custam 5 euros.

Questão

Indique a ordem dos testes e os valores de fronteira que devem ser verificados.

Comportamento Esperado: Estacionamento

horas	custo	caso
-1	—	entrada inválida
0	0	nenhuma permanência
1	3	até duas horas
2	3	fronteira incluída
3	5	mais de duas horas

Uma Solução Possível: Estacionamento

Problema: determinar o custo a partir do número inteiro de horas, distinguindo entradas inválidas.

```
1  if horas < 0:
2      print("horas invalidas")
3  elif horas == 0:
4      custo = 0
5      print(custo)
6  elif horas <= 2:
7      custo = 3
8      print(custo)
9  else:
10     custo = 5
11     print(custo)
```

Exercício: Simplificar Condições

```
1 if idade >= 18:
2     if tem_bilhete == True:
3         entrada = "permitida"
4     else:
5         entrada = "recusada"
6 else:
7     entrada = "recusada"
```

Questão

Escreva uma condição única que produza exatamente o mesmo resultado.

Resposta: Simplificar Condições

```
1 if idade >= 18 and tem_bilhete:
2     entrada = "permitida"
3 else:
4     entrada = "recusada"
```

- `tem_bilhete` já é um valor booleano; não precisa de `== True`.
- Os dois caminhos de recusa podem ser reunidos no mesmo `else`.

Exercício: Prever uma Expressão Condicional

```
1 x = 7
2 y = 10
3 maior = x if x > y else y
4 empate = x == y
```

Questão

Quais são os valores finais de `maior` e `empate`? Repita a previsão para `x=10, y=10`.

Resposta: Expressão Condicional

- Com `x=7, y=10`: `maior == 10` e `empate == False`.
- Com `x=10, y=10`: o ramo depois de `else` produz `maior == 10`, e `empate == True`.
- A expressão condicional escolhe um valor; a comparação produz outro valor booleano independente.

Leitura Final: Estrutura e Estado

Ao ler um programa com decisões, faça sempre três perguntas:

1. Que valores constituem o estado antes da condição?
2. Que ramo é executado e que variáveis altera?
3. Que estado chega à primeira instrução depois da alternativa?

Conclusão

Hoje vimos

- estado das variáveis durante a execução
- composição de instruções em programas estruturados
- condições e caminhos alternativos de execução

A seguir

- funções simples como forma de nomear cálculos
- algoritmos numéricos iniciais
- decomposição de problemas em passos testáveis